

实验四 存储管理

一、实验目的

- 1.通过模拟内存分配算法，加深对内存分配、回收算法的理解。
- 2.通过模拟页面替换算法，加深对页面替换算法的理解。
- 3.掌握页面替换算法中缺页中断率的计算方法

二、实验内容

1. 模拟实现 First Fit 和 Best Fit 算法。
2. 模拟实现 FIFO 和 LRU 算法
3. 计算 FIFO 和 LRU 算法的缺页中断率

三、实验要求

I.内存分配算法

1. 设计程序模拟内存动态分区的内存管理方法。内存空闲区采用空闲分区表进行管理，采用 First Fit/Best Fit 算法从空闲分区表中寻找空闲区进行分配，内存回收时考虑与相邻空闲区的合并。
2. 假定系统的内存共 640K，初始状态为操作系统本身占用 40K。t1 时刻，为进程 A、B、C 分配 80K、60K、100K 内存空间；t2 时刻进程 B 完成；t3 时刻为进程 D 分配 50K 的内存空间；t4 时刻进程 C、A 完成；t5 时刻进程 D 完成。要求编程分别输出 t1、t2、t3、t4、t5 时刻内存的进程情况和内存空闲区的状态。

II.页面替换算法

1.实验用到的数据结构

(1) 页表 (Page Table): 操作系统进行分页内存管理的数据结构，指明每个进程虚拟页和物理页之间的对应关系，每个进程有自己的一个页表。

(2) 页框 (Page Frame): 操作系统为每个进程分配的内存页面。

2.页面替换

在请求分页内存管理中，每个进程仅加载部分页进入内存，因此，系统需要查询

页表才能得知某页是否在内存中。当进程所需页面不在内存中时要产生缺页中断（page fault）信号。处理器接收到缺页中断信号，中断处理程序先保存现场，转入缺页中断处理程序。该程序通过查找页表，得到该页所在外存的物理块号。如果此时内存未满，能容纳新页，则启动磁盘 I/O 将所缺之页调入内存；如果内存已满，则需按某种替换算法从内存中选出一页准备换出，将所缺的页调入内存覆盖该页。这就是请求分页管理中的页面替换，也称为页面淘汰。

3.模拟实现

（1）先进先出页面替换算法（First in First Out）

（2）最近最久未使用页面替换算法（Least Recently Used）

4.缺页中断率（page fault rate）的计算

计算公式为：缺页中断率=缺页中断次数÷页面访问总次数×100%

5.指令序列的生成

（1）通过随机函数 rand（）产生一个指令执行序列，其中指令总数为 total_s，指令序列号为 0~total_s-1，指令总执行次数为 total_e。指令按下述原则执行：

①50%的指令是按顺序执行的。

②25%的指令是均匀分布在前地址部分的。

③25%的指令是均匀分布在后地址部分的。

（2）具体实现方法：

①在[0, total_s) 的指令地址之间随机选取指令执行起点 m1。

②顺序执行下一条指令，即执行地址为 m1+1 的指令。

③在前地址[0, m1) 中随机选取一条指令并执行，该指令的地址为 m2。

④顺序执行下一条指令，即执行地址为 m2+1 的指令。

⑤在后地址[m2, total_s-1) 中随机选取一条指令并执行，该指令的地址为 m3。

⑥顺序执行下一条指令，即执行地址为 m3+1 的指令。

⑦重复步骤①~⑥，直到 total_e 次执行为止。

6. 页面访问序列（reference string）的生成

按照指令的执行序列将随机生成的指令一次转化为需要访问的虚页序列，其中指令总数为 $total_s$ ，指令序列号为 $0 \sim total_s - 1$ ，程序虚页大小为 p_size ，即每一虚页能够存放的指令数为 p_size ，因此，用户指令组成的虚拟页面总数=指令总数/页面大小，即 $(total_s/p_size)$ 页，对应页号 $0 \sim total_s/p_size - 1$ 。具体如下所示：

第0条 ~ 第 $p_size - 1$ 条指令对应第 0 页；

第 p_size 条 ~ 第 $2*p_size - 1$ 条指令对应第 1 页；

.....

第 $total_s - p_size$ 条 ~ 第 $total_s - 1$ 条指令对应第（总页数 - 1）页。

7. 模拟程序输入输出要求

（1）输入 $total_s$ ， $total_e$ ， p_size ，内存中的页框数量（用 pf_num 表示）

（2）输出页面访问序列，FIFO 算法的缺页次数、缺页中断率，LRU 算法的缺页次数和缺页中断率。